The Journal of Open Source Software

FlapKine – A Simulation Toolkit for the Kinematics of Flapping-Wing Micro Aerial Vehicles

Kalbhavi Vadhi Raj [1], **Raj Kiran Sangoju**[2], **and Nipun Arora** [2]

1 Department of Electrical Engineering, Indian Institute of Technology Jodhpur, India **2** Department of Mechanical Engineering, Indian Institute of Technology Jodhpur, India

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 24 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

FlapKine is a modular, PyQt5-based application designed for 3D visualization of forward and inverse kinematics in flapping-wing systems. The application is implemented in Python with a hybrid PyQt5 + VTK architecture. Its hybrid architecture allows it to operate as both an intuitive graphical user interface (GUI) and a Python library, enabling researchers to simulate and animate flapping-wing motion either interactively or through code using FlapKine's core classes. Flapkine enables users to load and animate STL wing meshes with translation, rotation, and flexibility transformations, enabling visibly accurate animation of the flapping-wing mechanism. It also enables users to quickly tweak video animation configurations like video resolution, camera and light properties. FlapKine generates video animations in situ, this reduces the disk I/O overhead, enabling faster animation rendering. FlapKine is capable of performing inverse kinematics calculation of Euler rotation angles from the 3-D position time series of just four selected points on the wing plane. This inverse kinematics calculation is well detailed in FlapKine Docs. FlapKine also has the option to save the STL file at each time step, these STL files can be later on used for high-fidelity computational fluid dynamics (CFD). CFD platforms such as OpenFOAM(Jasak, 2009) and ANSYS(Ansys Inc., 2025) are equipped to import STL files for mesh generation and subsequent flow analysis. In OpenFOAM, utilities like snappyHexMesh utilize STL inputs to create volume meshes that conform to complex geometries, enabling accurate simulations of aerodynamic forces such as lift and drag. Similarly, ANSYS tools, including ICEM CFD and SpaceClaim, support STL file imports for mesh generation and geometry preparation, facilitating seamless integration into the CFD workflow.

Statement of Need

Insects and birds exhibit exceptional maneuverability and fine-grained flight control, which has inspired the development of bio-inspired Micro Aerial Vehicles (MAVs). These MAVs, characterized by their compact form factor and low weight, mimic the flapping-wing mechanisms observed in nature to achieve efficient and agile flight dynamics (Ahmed et al., 2024).

Research in flapping-wing MAVs demands precise kinematic modeling and visualization to enable the design of such vehicles, validate analytical frameworks, and process experimental motion capture data. However, current research workflows are often hampered by several limitations:

- **Steep programming barriers** — Visualizing even basic flapping-wing motion typically requires scripting complex pipelines or extending general-purpose 3D engines such as Blender (**?**). This often leads researchers to spend more time debugging and implementing visualization logic than actually analyzing flapping-wing behavior.

- **Fragmented toolchains** — Existing processes usually involve separate tools for different stages—e.g., DLTdv(Tyson et al., 2023) for extracting 3D trajectories from multi-view stereo video, custom scripts to convert trajectories into Euler angles, and external rendering software to animate those angles. This disjointed pipeline increases development overhead and complicates reproducibility.

- **I/O and performance bottlenecks** — When switching between isolated tools, researchers are forced to save and reload intermediate data files at every stage. Combined with single-threaded rendering loops, this file-based communication leads to high latency.

FlapKine fills this gap by offering a unified, GUI-driven environment that seamlessly integrates inverse kinematics, STL visualization, and in-memory rendering. It significantly lowers barriers for interdisciplinary teams—biomechanists, roboticists, and control engineers—who need both programmatic control and intuitive graphical interfaces.

Built from the ground up, **FlapKine** is:

- **Fully Open Source** — Licensed under a permissive open-source license to foster academic innovation and industry adoption.
- **Modular and Extensible** — Easily add custom transformation models, extend GUI windows, or swap back-end rendering logic to suit experimental or computational needs.
- **Research-Ready** — Purpose-built to support reproducibility and generate publication-quality visualizations.

Comparison to Existing Tools

To the best of our knowledge, there is no existing application that integrates all the functionalities provided by FlapKine in a single, cohesive platform. However, a few tools address related aspects within the domain of flapping wing micro air vehicles (MAVs):

- **MAV Study ver. 1.001**: MAV Study(Roccia et al., 2011) is an interactive computational code developed entirely in MATLAB to analyze all the kinematical parameters that characterize the flapping of the wings of a house fly, and to visualize the spatial trajectories of the material points of the wings coming from numerical simulations. However, MAV Study is tied to MATLAB, requiring a licensed environment to operate, and is specifically tailored for the housefly model. In contrast, **FlapKine** supports the import of arbitrary wings via STL files, making it applicable to any flapping wing system. It also enables wing translation and flexible deformation simulation—features not present in MAV Study—and is available without proprietary constraints.

- **DLTdv**(Tyson et al., 2023) is a motion analysis tool available as a MATLAB app, a standalone application, and source code. It supports both single and multi-camera setups, 3D reconstruction, and lens distortion correction. It has been effectively used for tracking wing tips and studying parameters such as angle of attack and camber deformation (Truong et al., 2012). Although DLTdv excels at extracting 3D positions from videos, it lacks a modular pipeline for solving inverse kinematics and computing Euler angles. This is where **FlapKine** acts as a complementary tool. It processes the 3D time series obtained from DLTdv, applies inverse kinematics to extract rotation parameters, and uses them to generate forward kinematic animations. Importantly, DLTdv data is only needed when performing inverse modeling—**FlapKine** can also generate simulations independently.

- **Blender**(Blender Foundation, 2025) is a powerful open-source 3D creation suite that covers modeling, animation, simulation, and rendering. While it can be used to simulate flapping wings, doing so requires manual scene setup, extensive scripting through its Python API, and a significant learning curve. **FlapKine**, by contrast, provides a GUI-driven platform tailored for flapping wing kinematics. It abstracts away the complexity of animation scripting and offers an accessible interface for researchers and engineers to simulate and visualize wing motion interactively and efficiently.

DRAFT

Overall **FlapKine** tries to combine most of the existing features in different softwares and provide a unified platform for flapping wing forward and inverse kinematics simulations.

Architecture and Design

FlapKine's design centers on a clear separation between **core components** and the **graphical user interface**, organized into:

1. Core Components

Main Object-Oriented Components

Component/Class	Description
`Object3D`	Represents a static 3D object. Stores STL mesh, name, and a transformation pipeline. Applies translation, rotation, and deformation — useful for simulating flexible or articulated bodies.
`Sprite`	Encapsulates time-series motion data for a single `Object3D`. Stores dynamic states such as the origin of the body frame, Euler angles, and deformation across frames.
`Scene`	Aggregates multiple `Sprite` instances. Used to simulate complex systems with multiple moving components (e.g., insect wings or robotic limbs).

These classes are modular and can be used directly in Python scripts or through the Flapkine GUI.

DRAFT

The Journal of Open Source Software

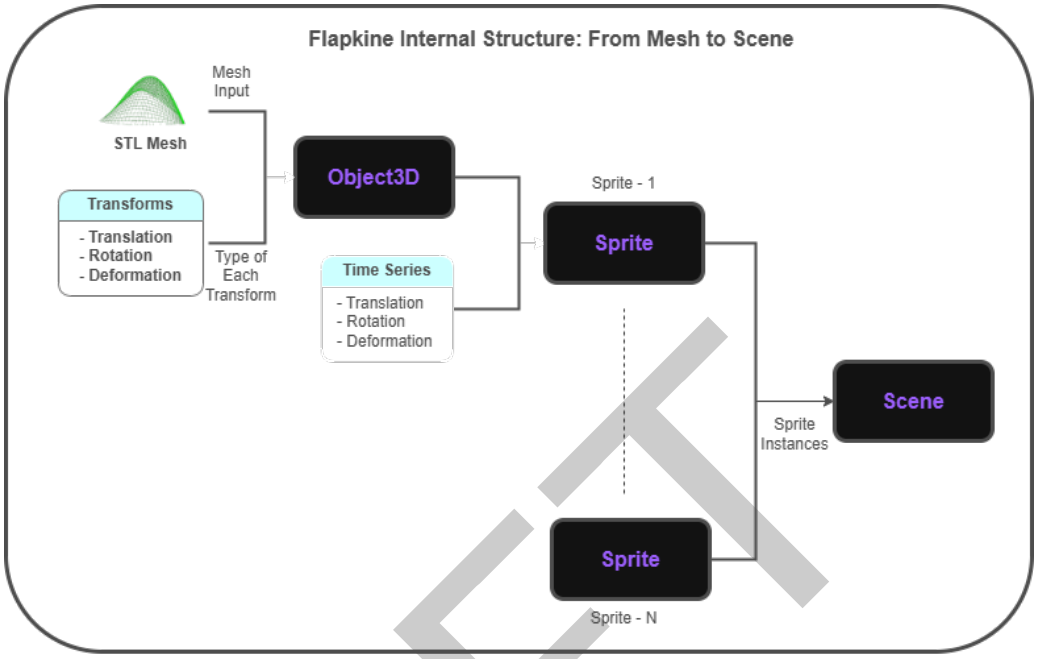

Figure 1: Object-oriented structure of the FlapKine backend showing how `Object3D`, `Sprite`, and `Scene` classes interact

Conceptual Analogy: A 3D Stage

To intuitively understand Flapkine's architecture, imagine the simulation as a **3D stage**:

- The **Scene** is the overall 3D space — it's the "world" or environment where everything happens. It contains multiple actors (objects), lights, and camera perspectives.
- An **Object3D** is a static 3D model — like a prop or structure on the stage. It knows its shape (via STL mesh), name, and transformation capabilities.
- A **Sprite** is an **Object3D in motion** — it represents that object undergoing transformations over time (translation, rotation, deformation). It's the actor performing on stage.
- The **Scene** is thus a **collection of Sprites**, each of which combines an Object3D and its motion sequence (e.g., Euler angle time series, translation time series, etc.).

Together, the structure looks like this:

```
Scene
├── Sprite 1 → (Object3D + Motion Time Series)
├── Sprite 2 → (Object3D + Motion Time Series)
│      .
│      .
│      .
└── Sprite N → (Object3D + Motion Time Series)
```

Each level in the structure adds more abstraction and control:

- `Object3D`: Static geometry + transformation logic.
- `Sprite`: adds time-based behavior
- `Scene`: Combines multiple `Sprite` instances into one coordinated simulation.

This layered structure makes it easy to manage and visualize multiple interacting elements in a biomimetic system, such as both wings of an insect etc.

DRAFT

2. Graphical User Interface (GUI)

FlapKine offers GUI windows for interactively creating and managing each of the object-oriented components discussed earlier (`Object3D`, `Sprite`, and `Scene`). This modular interface enables users—whether researchers, engineers, or students—to easily construct, simulate, and visualize complex biomechanical or robotic systems without needing to write code.

Below is a summary of all the windows provided by the FlapKine interface:

Application Windows Overview

Window	Description
Main Window	Entry point for opening or creating projects
Project Editor Window	View STL, select points, preview animations
Project Creator Window	Setup scenes, adjust render settings
Scene Creator Window	Compose scenes using sprites
Sprite Creator Window	Define object properties and orientation
Inverse Kinematics Window	For solving Inverse Kinematics problems

Each window is designed to operate both independently and as part of a larger simulation workflow, ensuring flexibility across a wide range of use cases.

Modeling Flow

FlapKine is a GUI based modelling framework, When application it offers to either open a initialised project or create a new project. We will go through the workflow behind creating a new project and opening a new project with the help of an example. See Example 1 - Documentation for detailed information.

1-DOF Flapping Wing

This example simulates a single-axis wing rotation (z-axis) with STL visualization.

Files

Download Example Project (project.zip)

- **`project.zip`**: Contains everything needed to run and create this example from scratch.
  - `1_DOF_1/`: Project folder
  - `resources/`: STL files, angle CSVs

Folder Structure:

```
project.zip
  ├── 1_DOF_1/
  │   ├── scene.pkl                # Scene Class object saved as a pickle file
  │   ├── config.json              # Configuration file with simulation and rendering s
  │   └── data/                    # Output directory for generated frames or videos
  │
  └── resources/
      ├── angles/
      │ ├── alpha_data.csv         # Rotation about the x-axis (all zeros)
      │ ├── beta_data.csv          # Rotation about the y-axis (all zeros)
      │ └── gamma_data.csv         # Rotation about the z-axis (time-series values)
      │
```

DRAFT

```
├── stl/
│   └── wing.stl                # 3D mesh of the wing
│
└── angle_plot.png              # Plot of the rotation angles over time
```

STL Orientation

- The x-axis aligns with the wing span.
- The y-axis aligns with the wing chord.
- The z-axis corresponds to the wing thickness.

Method 1: Load Provided Project

1. Extract `project.zip`.
2. Open **FlapKine** → **Load Project** → select `1_DOF_1/`.
3. Click **Render** to generate video output in `data/videos/`.

Method 2: Recreate From Scratch

1. Open FlapKine → **New Project** → choose folder and name → **Save**
2. In **Project Creator Window**, load default config:
   - Camera: `(0, 0, 50)`, Focal: `(0, 0, 0)`, Up: `(0, 1, 0)`
   - Light: Pos `(0, 0, 20)`, Energy: `1`
   - Enable STL export if needed
3. Click **Create** under Import Scene → opens **Scene Creator**
4. Click **Add** → **Sprite 1** → **Create** → opens **Sprite Creator**
5. In Sprite Creator:
   - Load `wing.stl` from `resources/stl/`
   - Set rotation Transform:
     - Type: `Euler_angles`, Order: `ZYX`
     - Angles: load `gamma_data.csv`, `beta_data.csv`, `alpha_data.csv` in Angle I, Angle II and Angle III respectively
6. Click **Finish**, then **Import Scene**
7. Back in Project Creator → click **Create Project**

This will reproduce the setup from `1_DOF_1/`

References

Ahmed, F., Kalbhavi, V. R., & Arora, N. (2024). A kinematic analysis of flow dynamics and aerodynamic performance in the clap-and-fling motion. *Physics of Fluids*, *36*(10), 101914. https://doi.org/10.1063/5.0232140

Ansys Inc. (2025). *ANSYS multiphysics simulation software*. https://www.ansys.com/.

Blender Foundation. (2025). *Blender – a 3D modelling and rendering package*. Blender Foundation. https://www.blender.org

Jasak, H. (2009). OpenFOAM: Open source CFD in research and industry. *International Journal of Naval Architecture and Ocean Engineering*, *1*, 89–94. https://doi.org/10.3744/JNAOE.2009.1.2.089

DRAFT

Roccia, B. A., Preidikman, S., Massa, J. C., & Mook, D. T. (2011). Development of a kinematical model to study the aerodynamics of flapping-wings. *International Journal of Micro Air Vehicles*, *3*(2), 61–88. https://doi.org/10.1260/1756-8293.3.2.61

Truong, T. V., Le, T. Q., Byun, D., Park, H. C., & Kim, M. (2012). Flexible wing kinematics of a free-flying beetle (rhinoceros beetle trypoxylus dichotomus). *Journal of Bionic Engineering*, *9*(2), 177–184. https://doi.org/10.1016/S1672-6529(11)60113-3

Tyson, H. L., Dickerson, A. K., Muijres, F. T., & Pieters, R. P. M. (2023). Tracking the body, wing, and leg kinematics of moving mosquitoes. *Cold Spring Harbor Protocols*, *2023*(2), pdb.prot107928. https://doi.org/10.1101/pdb.prot107928

DRAFT